

ЛР8 (3 пары / 6 часов)

«Единая валидация данных: правила для Title/Description. Сообщения об ошибках. Подготовка к тестированию»

Что уже есть

- CRUD + JSON хранение + import/export/backup
- `Title` проверяется только на "не пустое"
- `Description` есть, но правил почти нет

Что делаем в ЛР8

Создадим **единые правила валидации** (чтобы одинаково проверять везде):

- при добавлении
 - при редактировании
 - при импорте
-

0) Итог ЛР8 (что должно работать)

✓ Есть отдельный класс `TaskValidator`

✓ Правила:

- Title **обязателен**
- Title длина **1–50**
- Description длина **0–300** (можно пусто)
 - ✓ Ошибки показываются **понятно** (не "Exception", а "Ошибка: ...")
 - ✓ Валидация применяется:
- Add
- Update

- Import
-

1) Теория (важная, развернутая)

1.1. Что такое валидация

Валидация — это проверка данных перед тем как сохранить их.

Если валидации нет:

- в файл попадёт мусор
- потом будет невозможно нормально показывать/искать/сортировать
- программа начнёт “ломаться” в неожиданных местах

1.2. Почему делаем отдельный класс Validator

Новички часто пишут проверки везде:

- в Add
- в Update
- в Import

И потом правила расходятся (в одном месте 50 символов, в другом 100).

Поэтому делаем **одно место**:

- `TaskValidator.Validate(...)`

1.3. Как лучше возвращать ошибки новичкам

Не надо “страшных” исключений пользователю.

Схема:

- Validator возвращает **текст ошибки** (string) или `null` (если всё ок)
- App показывает “Ошибка: ...”

Это очень понятная схема для 1 курса.

2) Практика. Пара 1: создаём TaskValidator в Core

Шаг 1. Открыть проект Core

Visual Studio → TaskTracker.Core

Шаг 2. Создать папку Validation

1. Правой кнопкой по TaskTracker.Core
 2. Add → New Folder
 3. Название: Validation
-

Шаг 3. Создать класс TaskValidator.cs

1. Правой кнопкой по папке Validation
 2. Add → Class...
 3. Название: TaskValidator.cs
 4. Add
-

Шаг 4. Вставить код TaskValidator

Откройте TaskValidator.cs и вставьте:

```
using TaskTracker.Core.Models;

namespace TaskTracker.Core.Validation;

public static class TaskValidator
{
    public const int TitleMaxLength = 50;
    public const int DescriptionMaxLength = 300;
}
```

```
// Возвращает null если всё хорошо, иначе текст ошибки
public static string? Validate(TaskItem task)
{
    if (task is null)
        return "Задача не должна быть пустой (null).";

    // Title
    if (string.IsNullOrEmpty(task.Title))
        return "Название (Title) обязательно.";

    var title = task.Title.Trim();
    if (title.Length > TitleMaxLength)
        return $"Название слишком длинное. Максимум {TitleMaxLength} символов.";

    // Description
    var desc = (task.Description ?? "").Trim();
    if (desc.Length > DescriptionMaxLength)
        return $"Описание слишком длинное. Максимум {DescriptionMaxLength} символов.";

    return null;
}
```

Шаг 5. Сборка

Build → Build Solution → Build succeeded.

Шаг 6. Коммит №1

Сообщение:

- feat: add TaskValidator with title/description rules

Commit All → Push.

3) Практика. Пара 2: подключаем Validator в TaskService (Add/Update)

Шаг 7. Открыть TaskService.cs

TaskTracker.Core → Services → TaskService.cs

Шаг 8. Подключить using Validator

Сверху файла добавьте:

```
using TaskTracker.Core.Validation;
```

Шаг 9. Изменить Add: сначала создаём TaskItem, затем валидируем

Найдите метод `Add(string title)` и замените его на:

```
public TaskItem Add(string title)
{
    var task = new TaskItem
    {
        Id = _nextId,
        Title = title ?? "",
        Description = "",
        Status = TaskStatus.New
    };

    var error = TaskValidator.Validate(task);
    if (error != null)
        throw new ArgumentException(error);

    task.Title = task.Title.Trim();

    _tasks.Add(task);
}
```

```
    _nextId++;

    return task;
}
```

Шаг 10. Изменить Update: валидируем через Validator

Найдите `Update(int id, string newTitle, string newDescription)` и замените на:

```
public TaskItem Update(int id, string newTitle, string newDescription)
{
    var task = GetExisting(id);

    // временно присваиваем новые значения
    task.Title = newTitle ?? "";
    task.Description = newDescription ?? "";

    var error = TaskValidator.Validate(task);
    if (error != null)
        throw new ArgumentException(error);

    // нормализуем (Trim)
    task.Title = task.Title.Trim();
    task.Description = (task.Description ?? "").Trim();

    return task;
}
```

Шаг 11. Сборка

Build → Build Solution → Build succeeded.

Шаг 12. Коммит №2

Сообщение:

- `feat: use TaskValidator in TaskService add/update`

Commit All → Push.

4) Практика. Пара 3: валидация при Import + проверка сценариев + отчёт

Шаг 13. Обновить Import в Program.cs (валидация всех импортируемых задач)

Откройте `TaskTracker.App → Program.cs`

Добавьте using:

```
using TaskTracker.Core.Validation;
```

Теперь найдите ваш блок `input == "11"` (Import).

После строки:

```
var importedTasks = importStorage.Load();
```

вставьте проверку:

```
// Валидация импортируемых задач
for (int i = 0; i < importedTasks.Count; i++)
{
    var t = importedTasks[i];

    var error = TaskValidator.Validate(t);
    if (error != null)
    {
        Console.WriteLine($"Ошибка импорта: задача #{i + 1} н
```

```

    е прошла проверку: {error}");
        Console.WriteLine("Импорт отменён.");
        continue;
    }
}

```

⚠ Важно: `continue;` внутри цикла не остановит импорт целиком.

Для новичков проще сделать так:

Замените весь блок на **правильный вариант**:

```

bool ok = true;

for (int i = 0; i < importedTasks.Count; i++)
{
    var t = importedTasks[i];
    var error = TaskValidator.Validate(t);

    if (error != null)
    {
        Console.WriteLine($"Ошибка импорта: задача #{i + 1} н
е прошла проверку: {error}");
        ok = false;
        break;
    }
}

if (!ok)
{
    Console.WriteLine("Импорт отменён.");
    continue;
}

```

Теперь импорт отменится корректно.

Шаг 14. Проверка (F5) — сценарии

Запустите программу (F5) и проверьте:

Сценарий А: Title пустой

1. Добавить задачу (1)
2. Нажать Enter на пустом вводе
3. Должно быть: "Ошибка: Название обязательно"

Сценарий В: Title слишком длинное

1. Добавить задачу (1)
2. Вставить строку из 60 символов
3. Должно быть: "Максимум 50"

Сценарий С: Description слишком длинное (через редактирование)

1. Добавить задачу
2. Редактировать (5)
3. Ввести Description 400 символов
4. Должно быть: "Максимум 300"

Сценарий D: импорт плохого файла

1. Создайте файл JSON, где у задачи Title пустой
2. Импорт (11)
3. Должно написать ошибку импорта и **не заменить** задачи

Шаг 15. Отчёт `docs/LR8_Report.md`

Создайте файл `docs/LR8_Report.md` :

```
# Отчёт ЛР8 – TaskTracker
```

```
## Что сделано
```

- Создан TaskValidator с правилами:
 - Title обязателен, максимум 50 символов
 - Description максимум 300 символов
- Валидация подключена в:
 - Add
 - Update
 - Import (перед заменой задач)

```
## Как проверить
```

- 1) Добавить пустой Title -> ошибка
- 2) Добавить Title > 50 -> ошибка
- 3) Редактировать Description > 300 -> ошибка
- 4) Импортировать JSON с пустым Title -> импорт отменён

Шаг 16. Коммит №3

Сообщение:

- `feat: validate imported tasks and add LR8 report`

Commit All → Push.

5) Что сдавать (чек-лист)

✓ ссылка на репозиторий

✓ `docs/LR8_Report.md`

✓ видео/скрин:

- попытка добавить пустой Title (ошибка)
- попытка импортировать плохой JSON (ошибка)

✓ минимум 3 коммита за ЛР8

6) Оценивание

"3" — есть TaskValidator и он работает при Add

"4" — работает Add + Update + Import с валидатором

"5" — хорошие сообщения об ошибках + проверены сценарии + аккуратные КОММИТЫ
